Assignment 6: Database Views and Normalization

Part I: Database Views

B. Creating Views

View 1: view_limited_recipes (Limited-Access View)

This view hides sensitive information from the Users and Recipes tables. It strips out the user password, as well as internal caching fields like source_api, last_fetched, and cache_priority that have no need to be exposed to a regular user. It only returns recipe data associated with a user's favorites.

```
CREATE VIEW view_limited_recipes AS
SELECT
   r.recipe_id,
   r.recipe_name,
   r.instructions,
   r.image_url,
   r.is_favorited,
   u.user_id,
   u.name AS user_name
FROM Recipes r
JOIN Favorites f ON r.recipe_id = f.recipe_id
JOIN Users u ON f.user_id = u.user_id;
```

Expected result: A table showing recipe details and the user who favorited them, with no password or cache data visible.

View 2: view_user_favorites (Role-Based View)

This view is intended for a user-facing role. It only shows recipes that have been favorited (is_favorited = 1), along with the user's rating and notes. An employee or app-layer user querying this view only sees what is relevant to the current user's saved recipes.

```
CREATE VIEW view_user_favorites AS
SELECT
   f.favorite_id,
   f.user_id,
   u.name AS user_name,
   r.recipe_name,
   f.rating,
   f.notes,
   f.date_added
FROM Favorites f
JOIN Users u ON f.user_id = u.user_id
JOIN Recipes r ON f.recipe_id = r.recipe_id
WHERE r.is_favorited = 1;
```

Expected result: A filtered list of favorited recipes per user, showing only what a logged-in user would need to see. No sensitive data is included.

C. Simulating Role-Based Access (Written SQL Only, No Execution)

Granting privileges on a view to a specific user:

This grants a read-only user access to the limited-access view. The user can query the view but cannot see the underlying tables or modify any data.

```
-- Grant read access to the limited view for a read-only user
GRANT SELECT ON liammcke_inventory_recipes.view_limited_recipes
    TO 'readonly_user'@'localhost';

-- Grant read access to the role-based view for an app user
GRANT SELECT ON liammcke_inventory_recipes.view_user_favorites
    TO 'app_user'@'localhost';
```

Creating a role and assigning it to a user:

```
-- Create the role
CREATE ROLE 'employee_role';

-- Grant the role access to the role-based view
GRANT SELECT ON liammcke_inventory_recipes.view_user_favorites
    TO 'employee_role';

-- Assign the role to a user
GRANT 'employee_role' TO 'testuser'@'localhost';

-- Set it as the default role so it activates on login
SET DEFAULT ROLE 'employee_role' TO 'testuser'@'localhost';
```

This will not be used in our database; this is just being used to show how to create and assign a user to a role.

Part II: Database Normalization

A. Normalization Analysis

The database was reviewed against 1NF, 2NF, and 3NF rules. Two violations were found.

1NF Compliance

All tables in the database are 1NF compliant. Every table has a primary key, all columns contain a single data type, and there are no repeating groups or multi-valued attributes. Row ordering is not used to convey any meaning.

2NF / Design Violation: Nutritional_Values Table

The Nutritional_Values table violates good relational design by mixing two different entity types into a single table using nullable foreign keys and a discriminator column (entity_type). Each row either has an ingredient_id or a recipe_id set, but never both. The entity_type enum ('ingredient' or 'recipe') is a workaround for the fact that the table is trying to represent two separate relationships at once.

This is a violation because the non-key columns (calories, protein, carbs, etc.) do not have a consistent functional dependency. For ingredient rows, they depend on ingredient_id. For recipe rows, they depend on recipe_id. Mixing both into a single table with nullable foreign keys introduces unnecessary nulls and makes the schema harder to reason about.

Fix: Split into two separate tables

```
CREATE TABLE Ingredient_Nutrition (
   nutrition_id INT PRIMARY KEY AUTO_INCREMENT,
   ingredient_id INT NOT NULL,
   calories DECIMAL(8,2),
   protein DECIMAL(8,2),
   carbs DECIMAL(8,2),
   sugar DECIMAL(8,2),
   fat DECIMAL(8,2),
   FOREIGN KEY (ingredient_id) REFERENCES Ingredients(ingredient_id)
);

CREATE TABLE Recipe_Nutrition (
   nutrition_id INT PRIMARY KEY AUTO_INCREMENT,
   recipe_id INT NOT NULL,
   calories DECIMAL(8,2),
   protein DECIMAL(8,2),
   carbs DECIMAL(8,2),
   sugar DECIMAL(8,2),
   fat DECIMAL(8,2),
   FOREIGN KEY (recipe_id) REFERENCES Recipes(recipe_id)
);
```

3NF Violation: is_favorited in Recipes Table

The is_favorited column in the Recipes table is a transitive dependency. Whether a recipe is favorited is determined by the Favorites table, not by anything intrinsic to the recipe itself. Storing it redundantly in Recipes means that if a user removes a recipe from their favorites, the Favorites table and the Recipes table both need to be updated. If one gets updated and the other does not, the data becomes inconsistent.

This violates 3NF because is_favorited depends on the existence of a row in Favorites, not directly and solely on recipe_id.

Fix: Remove is_favorited from Recipes and derive it from Favorites

```
ALTER TABLE Recipes DROP COLUMN is_favorited;

-- If needed, create a view to replicate the behavior:
CREATE VIEW view_favorited_recipes AS
SELECT DISTINCT r.recipe_id, r.recipe_name
FROM Recipes r
JOIN Favorites f ON r.recipe_id = f.recipe_id;
```

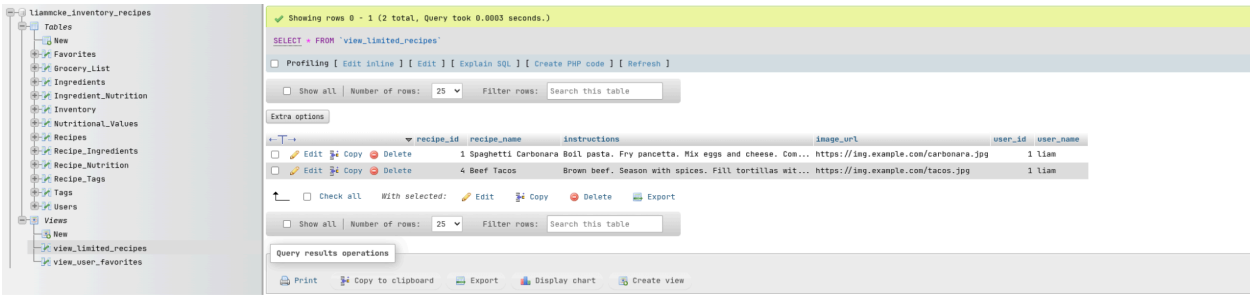
Part III: Documentation Summary

1. View SQL and Purpose

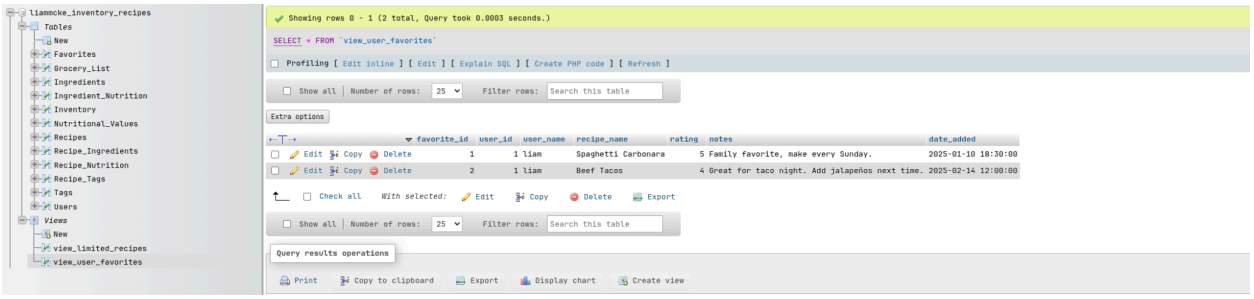
Both views are documented in Part I, Section B above with full SQL and explanations of their purpose and expected output. These SQLs are the view_limited_recipes and view_user_favorites.

2. Screenshots

View 1: limited recipes:



View 2: User favorites (with favorites notes):



3. Privilege Management SQL

The GRANT and CREATE ROLE statements are documented in Part I Section C above. Each command includes a comment explaining what it does and why.

4. Non-Conforming Elements Identified

- Nutritional_Values: Mixes ingredient and recipe nutrition data into one table using nullable foreign keys and a discriminator column. Non-key attributes do not depend on a single key.

- Recipes.is_favorited: Stores favorited status directly on a recipe when that information is already captured in the Favorites table. This creates a transitive dependency and risks data inconsistency.

5. Changes Made and Justification

Splitting Nutritional_Values into Ingredient_Nutrition and Recipe_Nutrition:

Separating these into two tables means every row in each table has a clear, single foreign key. There are no more nullable FK columns, no discriminator enum, and no ambiguity about what a row represents. Each table's non-key attributes depend fully and only on their respective primary key. This eliminates the nulls that existed before and makes the schema easier to query and maintain.

Removing is_favorited from Recipes:

Dropping this column removes a redundant piece of data that was already captured by the Favorites table. Deriving favorited status through a JOIN or a view instead means there is only one source of truth. This prevents the update anomaly where a user could remove a favorite but the Recipes table still shows is_favorited = 1, or vice versa. The view_favorited_recipes view replicates the original behavior without the redundancy.